PRÉSENTATION À L'ÉQUIPE QA

Test Installer 4D

Automatiser la checklist de release des installeurs 4D



Windows & macOS | 5 langues | 12 contrôles | 0 IA à l'exécution

Octobre 2026

SOMMAIRE

Au programme

1 **Pourquoi cette application ?**
Le besoin, la réponse, l'historique

2 **Avec quoi elle a été développée**
GitHub Copilot, langages et outils

3 **Utilisation sans Copilot**
Autonome, distribuable, reconstructible

4 **Fonctionnement**
Architecture, parcours, les 12 contrôles

5 **Résultats réels et limites**
Ce qu'elle a déjà trouvé, points d'attention

6 **Démo et questions**
L'application en direct

Pourquoi cette application ?

Avant : une checklist manuelle

- 12 contrôles à dérouler à la main pour chaque installeur
- … et dans chaque langue : EN, FR, DE, CS, JA
- Changer la langue de l'OS, installer, lancer 4D, vérifier signature, composants, plugins, ReadMe…
- Long, répétitif, risque d'oubli, résultats différents selon la personne

Avec Test Installer 4D

- Un formulaire et un bouton « Lancer la checklist »
- Les contrôles s'exécutent seuls, avec les preuves (hash, signatures, journaux)
- Un rapport standard : une colonne par langue, ✅ ❌ ⚠️ N/A
- Fini les gestes manuels : réseau et « À propos » automatiques, langue sans déconnexion sur Mac

12
contrôles automatisés

5
langues testées

7
langues du ReadMe analysées

2
plateformes : Windows et macOS

D’un agent Copilot à une application autonome

Début octobre

1

Agent Copilot

- Agent « Test installer 4D » dans GitHub Copilot (fichier .agent.md)
- La checklist est pilotée en conversation
- Copilot est nécessaire à chaque exécution

Jeudi 15/10

2

Application Windows

- « Peux-tu faire cet agent localement comme une app exécutable ? »
- C# / WinForms, checklist complète testée sur 4D 21.2
- Ajout du test ODBC

Vendredi 16/10

3

Version macOS

- Connexion à product-download.4d.com dans l’app Windows
- Icône, paquet zip Windows + kit macOS
- DMG macOS avec une vraie interface (SwiftUI)

Lundi 19/10

4

Fiabilisation

- Support de 4D 20 et plugin I.C. de référence
- Analyse automatique du ReadMe
- Tchèque (CS) au lieu de l’espagnol, bouton « Nouveau test »
- Versions Windows et macOS séparées

20 et 21/10

5

Zéro intervention

- Windows : « À propos » lu par OCR, test hors ligne automatique
- macOS : build lu dans le DMG, langue de 4D sans déconnexion
- macOS : réseau, Gatekeeper, Rosetta et relances GitHub automatiques

Développée du 15 au 21 octobre, en dialoguant avec GitHub Copilot.

Écrite par GitHub Copilot, pilotée par la QA

Outil	GitHub Copilot (application de bureau) avec l'agent personnalisé « Test installer 4D »
Modèle d'IA	Claude Opus 5.5
Méthode	Les besoins sont exprimés en français ; Copilot écrit le code, compile, lance les tests et corrige
Rôle de la QA	Définir la checklist, tester sur de vrais installeurs (4D 20.8, 20.9, 21.2), valider chaque résultat
Sans IDE	Compilateurs déjà présents sur les postes : csc.exe (Windows), swiftc (Command Line Tools macOS)

Exemples de demandes faites à Copilot

> *« Peux-tu faire cet agent localement comme une app exécutable ? »*

> *« Ajoute une case macOS, et télécharge depuis product-download.4d.com avec mon login. »*

> *« C'est du Terminal : fais-en une application avec une interface. »*

> *« Automatise l'analyse du ReadMe pour ne plus la tester à la main. »*

> *« Supprime la langue ES et ajoute le tchèque (CS). »*

> *« Automatise tous les points encore manuels dans l'application. »*

macOS : Swift + Bash, 100 % natif

Couche	Technologie	Fichier(s)	Rôle
Interface	Swift / SwiftUI	`TestInstaller4D.swift`	Formulaire, prérequis, tableau ✅/❌ en direct, journal, rapport
Moteur de checklist	Bash + outils macOS, AppleScript	`qa_mac.sh, dmg_info.sh`	hdiutil, codesign, spctl, networksetup, defaults, curl…
Analyse du ReadMe	Swift : PDFKit, NaturalLanguage, correcteur macOS	`readme_check.swift`	Lit le ReadMe.pdf et vérifie les 7 langues
« À propos », Gatekeeper	Swift : Accessibilité, Vision (OCR)	`about_check.swift`	Ouvre et lit la boîte « À propos », valide l'alerte « app téléchargée »
Tests dans 4D	Langage 4D	`QA_Local, QA_GitHub, QA_IC, QA_Exec, QA_About`	Composants, plugins, build, langue, 5 tests fonctionnels
Rapport	Bash + awk	`rapport_mac.sh`	HTML + Markdown, une colonne par langue, captures « À propos »
Construction	swiftc → lipo → codesign → hdiutil	`build_dmg.sh`	App universelle ARM + Intel, DMG ≈ 2,4 Mo ; notarisée si certificat Developer ID

Pourquoi pas Python ? macOS ne fournit pas d'interpréteur Python par défaut : une app Python aurait dû l'embarquer (dizaines de Mo). Swift donne une app native, légère et universelle, sans rien installer.

Windows : C# / .NET, compilé sans rien installer

Couche	Technologie	Fichier(s)	Rôle
Interface + moteur	C# (WinForms), .NET Framework 4.8	`TestInstaller4D.cs`	Formulaire, checklist, rapport
Téléchargement	Microsoft Edge WebView2	`Download.cs`	Connexion à product-download.4d.com, choix du build, téléchargement + I.C.
Analyse du ReadMe	C#	`ReadMe.cs`	Langue, paragraphes non traduits, version, encodage, liens
Système	PowerShell et API Windows	—	Langue et région, signature Authenticode, registre ; OCR Windows (« À propos »), pare-feu (test hors ligne)
Tests dans 4D	Langage 4D	projets de test (qaTest)	Composants, plugins, build, ODBC (SQL LOGIN)
Construction	csc.exe fourni avec Windows	`build.ps1`	Exe + 3 DLL WebView2 ; droits administrateur

Boutons clés de l'application Windows

⬇️ **Télécharger (product-download)…** 🔍 **Analyser le ReadMe** 🖌️ **Nouveau test (réinitialiser)**

Basculer / restaurer la langue **Générer le rapport**

🔒 Le mot de passe 4D reste dans la page de connexion 4D : l'application ne le lit pas et ne l'enregistre pas.

Copilot l’a écrite… mais n’est pas nécessaire pour l’utiliser

100 % autonome à l’exécution

Aucune IA · aucun compte GitHub · aucune licence Copilot

Distribuer

- **macOS :** TestInstaller4D.dmg (≈ 2,4 Mo), glisser l’app dans Applications ou lancer install.sh
- **Windows :** un zip (exe + 3 DLL WebView2), double-clic
- Se copie sur n’importe quel poste de test

Utiliser

- Formulaire + bouton « Lancer la checklist »
- Ou sans interface, dans le Terminal :

  `bash ~/TestInstaller4D/qa_mac.sh FR`
- Rapport HTML / Markdown à partager

Faire évoluer

- Code source livré avec l’application
- Recompiler :

  `bash macOS-app/build_dmg.sh`

  `build.ps1` (Windows)
- Modifiable avec ou sans Copilot

Internet n’est utile que pour : le téléchargement, les composants GitHub, la vérification des liens du ReadMe et le test « avec Internet ».

Agent Copilot ou application : deux outils complémentaires

	Agent Copilot « Test installer 4D »	Application Test Installer 4D
Prérequis	Licence GitHub Copilot + application de bureau	Windows 10/11 ou macOS 13+, rien d'autre
Lancement	Une demande en langage naturel	Un formulaire et un bouton
Exécution	L'IA choisit les commandes à chaque fois	Scénario fixe, identique à chaque exécution
Résultat	Tableau dans la conversation	Rapport HTML / Markdown + fichiers TSV par langue
Points forts	Souple : enquête sur un échec, s'adapte, prépare un bug	Rapide, reproductible, utilisable par toute l'équipe
Quand l'utiliser ?	Cas particuliers, analyse d'un ❌	Validation de chaque release, dans chaque langue

Prérequis et installation

Windows

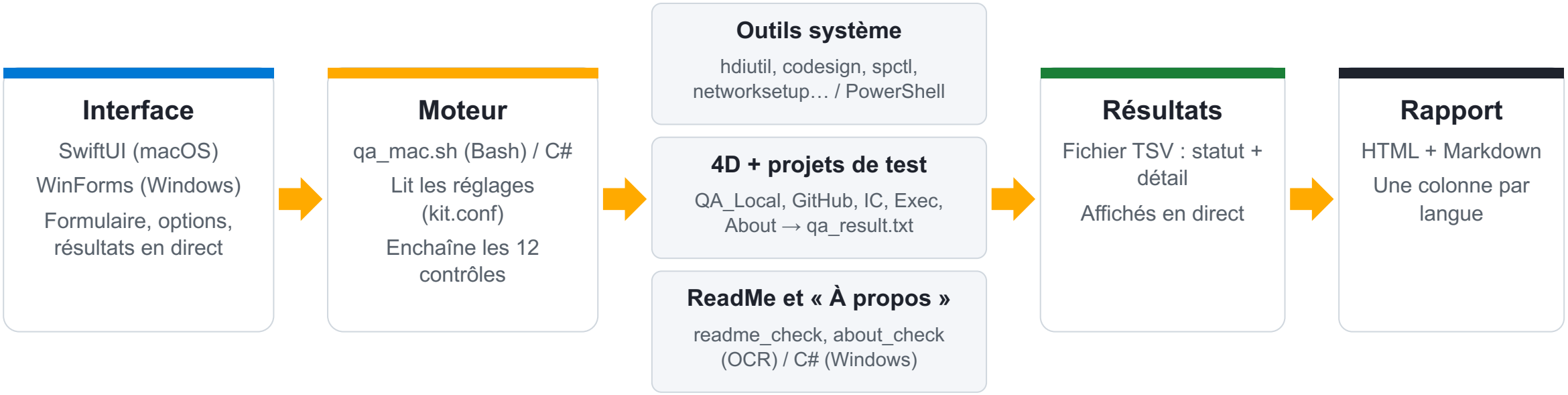
- Windows 10/11 x64
- .NET Framework 4.8 (déjà inclus dans Windows)
- Runtime Microsoft Edge WebView2 (présent avec Edge)
- Lancer TestInstaller4D.exe : droits administrateur demandés (langue de Windows, installation de 4D)
- Garder les 3 DLL WebView2 à côté de l'exe
- **Données :** `C:\4DQA` + dossiers results\ et proj\

macOS

- macOS 13 ou plus récent, Mac Apple Silicon ou Intel
- Installation : install.sh (app non notarisée) ; ouverture directe une fois notarisée
- Autoriser Accessibilité et Enregistrement de l'écran : boutons « Autoriser… » du panneau Prérequis
- Rosetta pour le test Rosetta : bouton « Installer… »
- Command Line Tools facultatifs : file et codesign remplacent lipo et stapler
- Compte administrateur : réseau coupé et rétabli automatiquement
- **Données :** `~/TestInstaller4D` et `~/4DQA_mac`

Dans les deux cas : l'installeur 4D se télécharge sur product-download.4d.com avec son compte 4D.

Architecture : une interface, un moteur, 4D en pilote automatique



Fichiers produits (macOS)

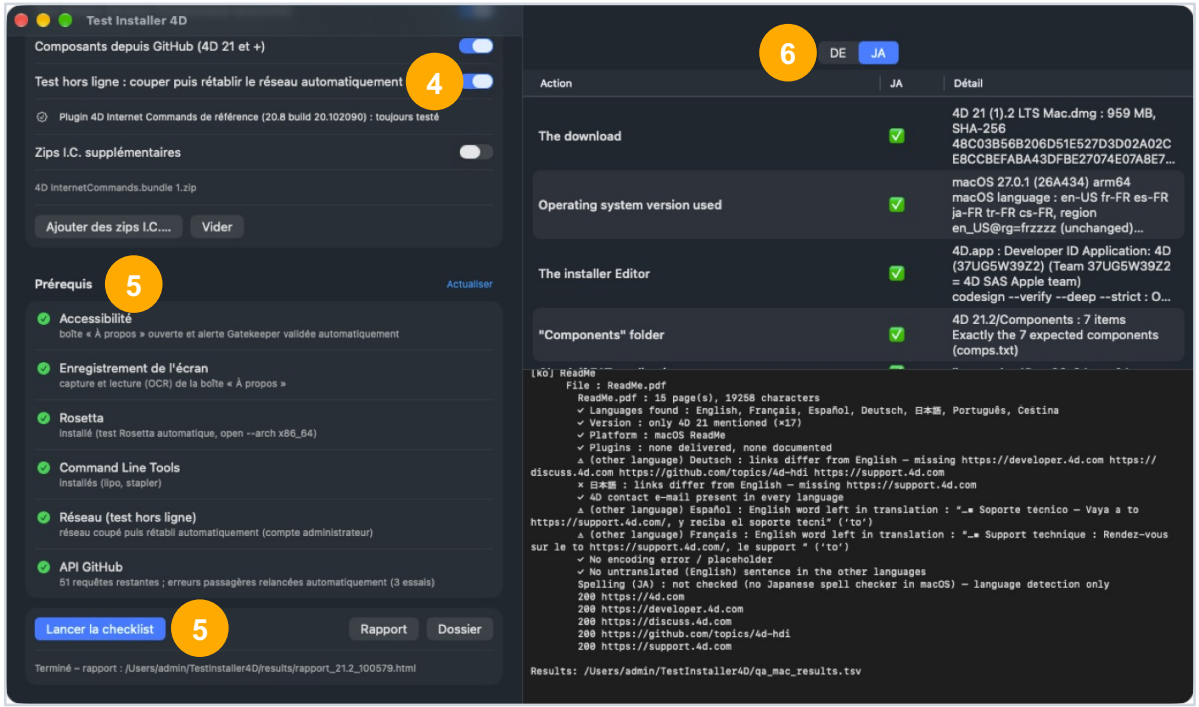
- `kit.conf` → derniers réglages saisis
- `results/<version>_<build>_<LANGUE>.tsv` → résultats par langue
- `results/<version>_<build>_<LANGUE>_about.png` → capture « À propos »
- `results/rapport_<version>_<build>.html / .md` → rapport final
- `~/4DQA_mac/<version>_<build>/` → copies de travail de 4D.app et projets

Pourquoi copier le kit ?

Le DMG est en lecture seule : au lancement, scripts et projets 4D sont copiés dans ~/TestInstaller4D. Réglages et résultats y sont conservés.

Le parcours : 7 étapes pour toutes les langues

1. Télécharger l'installeur sur product-download.4d.com (Windows : directement depuis l'app)
2. Ouvrir Test Installer 4D : le dernier DMG est détecté, version et build sont lus dans le DMG
3. Cocher les langues à tester (EN, FR, DE, CS, JA) : elles s'enchaînent
4. Options : test témoin, composants GitHub (4D 21+), test hors ligne automatique, zips I.C.
5. Prérequis au vert, puis « Lancer la checklist » : 4D s'ouvre plusieurs fois, aucune question
6. Suivre le tableau ✅/❌ en direct, une vue par langue
7. Ouvrir le rapport HTML : une colonne par langue, captures de la boîte « À propos »



macOS v3 après un test DE + JA : prérequis au vert, une vue par langue

Changer de langue : sur Mac, 4D est lancé dans chaque langue cochée (langue par application) : la langue du Mac ne change pas, aucune déconnexion. Sous Windows, l'app bascule la langue de Windows, puis déconnexion / reconnexion.

Les 12 contrôles de la checklist, automatisés

#	Contrôle	Comment c'est vérifié
1	**Version de l'OS**	Version exacte de l'OS et architecture (arm64 / x86_64)
2	**Téléchargement**	Fichier présent et non vide, taille, SHA-256, intégrité (hdiutil verify)
3	**Éditeur de l'installeur**	Signature 4D SAS : Authenticode (Windows) ; codesign + Team ID 37UG5W39Z2 (macOS)
4	**Numéro de version**	Info.plist / propriétés de l'exe, build renvoyé par 4D ; boîte « À propos » lue par OCR
5	**Dossier « Components »**	Exactement les 7 composants attendus (6 en 4D 20) ; manquants / en trop listés
6	**Plus de plugins livrés**	Aucun dossier Plugins, PLUGIN LIST = 0 dans 4D (N/A en 4D 20)
7	**4D Internet Commands**	Plugin de référence copié dans un projet : chargé, IT_SetTimeOut(30) sans erreur
8	**Chargement des composants**	Local + témoin + GitHub (4D 21+, jusqu'à 3 essais) : voir la page suivante
9	**Application FAT**	lipo -archs (ou file) = x86_64 arm64 (macOS)
10	**Exécution**	Gatekeeper, agrafage ; app en quarantaine lancée hors ligne (coupure automatique) puis en ligne
11	**Mode Rosetta**	4D lancé en Intel sur Mac ARM (open --arch x86_64) : processus « translated », sans plantage
12	**ReadMe**	Analyse automatique du PDF dans les 7 langues : voir plus loin

✅ **réussi** ❌ **échec (avec détail)** ⚠️ **à vérifier (si l'automatisation est impossible)** **N/A non applicable (avec raison)**
Windows : contrôle ODBC (SQL LOGIN) en plus ; FAT, Rosetta et agrafage sont N/A.

Chargement des composants : 3 scénarios

A · Local

- Projet 4D vierge, dependencies.json vide
- Les composants livrés sont copiés dans 4D.app/Contents/Components
- Tous doivent se charger et passer les 5 tests

B · Témoin

- Même projet, dossier Components renommé
- Seuls 4D Labels et 4D Report doivent rester
- Prouve que le test A utilise bien les composants livrés

C · GitHub (4D 21 et +)

- dependencies.json en `"version":"4d"` (« Suivre la version de 4D »)
- 7 composants téléchargés depuis github.com/4d
- Tags lus dans dependencies-lock.json ; 3 essais si erreur passagère

Les 5 tests exécutés dans 4D (méthode qaTest, avec ON ERR CALL)

`Length(SVG_New)` `Progress New` `cs.NetKit.OAuth2Provider` `cs.AIKit.OpenAI` `VP Font to object`

Résultat réel, 4D 21.2 : Local 9 composants chargés, 5 tests OK · Témoin OK · GitHub 7/7 (AIKit 21.8, NetKit 21.6, Progress 21.5, SVG 21.9, ViewPro 21.6, Widgets 21.5, WritePro Interface 21.2)

Analyse automatique du ReadMe (7 langues)



- ✓ Les 7 langues présentes (EN, FR, ES, DE, JA, PT, CS)
- ✓ Version : seulement « 4D <majeure> »
- ✓ Plateforme : le bon OS est cité
- ✓ Plugins : I.C. cité seulement s'il est livré
- ✓ Liens identiques à l'anglais et accessibles
- ✓ Une adresse @4d.com dans chaque langue
- ✓ Mots anglais oubliés, phrases non traduites
- ✓ Encodage, marqueurs TODO / XXX
- ✓ Orthographe (correcteur de macOS)

Règle : défaut dans la langue testée → ❌ ; dans une autre langue → ⚠️ (signalé sans échec).

Windows : ReadMe.cs vérifie la section de la langue testée (langue, non-traduit, version, encodage, liens).

Déjà trouvé dans les vrais ReadMe

- **4D 21.2, FR et ES :** mot anglais resté, « Rendez-vous sur le to https://support.4d.com/ »
- **4D 21.2, DE :** 4 liens de la version anglaise absents ; JA : lien support absent
- **4D 21.2 (Windows), JA :** copyright resté en anglais ; CS : lien du gestionnaire de dépendances absent
- **Propriétés du PDF :** titre périmé « Welcome to 4D v11 SQL Release 2 »
- **4D 20.9, CS :** fautes d'orthographe, ex. « documentaci » → « dokumentaci », « licenzování » → « licencování »

4D 20 ou 4D 21+ : la checklist s’adapte toute seule

Point contrôlé	4D 20	4D 21 et +
Composants attendus	6 (sans 4D AIKit)	7
Emplacement des composants	Dans 4D.app/Contents/Components	Dossier Components à la racine du DMG
Plugins livrés	Oui (4D Internet Commands) → N/A	Aucun attendu → contrôlé
Composants depuis GitHub	N/A (fonction absente en 4D 20)	Testé
Test AIKit dans 4D	Retiré	Testé

Choix automatique

- La version majeure (lue dans le DMG) sélectionne le bon scénario.
- En 4D 20, l’option GitHub est grisée dans l’interface, avec l’explication.

Plugin 4D Internet Commands de référence

- Plus mis à jour par 4D : le dernier publié (livré avec 4D 20.9) est figé dans le kit.
- Toujours testé avec la version de 4D contrôlée ; d’autres zips I.C. peuvent s’ajouter.

Exemple réel : 4D 21.2 LTS sur macOS ARM, en DE et JA

Run du 21/10/2026 lancé depuis l'app v3 · macOS 27.0.1 (26A434) · DMG 959 Mo, SHA-256 48C03B56…D21E · DE puis JA, sans déconnexion

Action	DE	JA	Détail
Operating system version used	✅	✅	macOS 27.0.1 (26A434) arm64 ; langue du Mac inchangée, 4D lancé en de-DE puis ja-JP
The download	✅	✅	959 Mo, SHA-256 relevé, hdiutil verify OK
The installer Editor	✅	✅	Developer ID Application: 4D (37UG5W39Z2) = équipe Apple de 4D SAS
Version number	✅	✅	Build 100579 lu dans le DMG = build de 4D ; « À propos » lu par OCR, dans la bonne langue
"Components" folder	✅	✅	Exactement les 7 composants attendus
No more plugins provided	✅	✅	Aucun plugin livré ; PLUGIN LIST = 0
Download last 4D I.C.	✅	✅	Plugin de référence chargé dans 4D 21.2, IT_SetTimeOut(30) OK
Load of components	✅	✅	Local : 5 tests OK · Témoin OK · GitHub : 7/7
Check if FAT application	✅	✅	lipo -archs : x86_64 arm64
Execution of application	✅	✅	Réseau coupé (4 services) puis rétabli ; alerte Gatekeeper validée automatiquement
Execution in 'Rosetta' mode	✅	✅	open --arch x86_64 : processus « translated », sans plantage
ReadMe	❌	❌	DE : 4 liens de l'anglais absents · JA : lien support.4d.com absent (vrais défauts du ReadMe)

À retenir : deux langues à la suite, sans aucune intervention. Le seul ❌ est un vrai défaut du ReadMe 4D, à remonter.

Ce que la v3 a réglé, ce qui reste

Réglé par la v3 (macOS, 21/10)

- ✓ Build : lu dans le DMG (Info.plist), plus rien à saisir
- ✓ Boîte « À propos » : ouverte et lue par OCR (version, build, langue), capture dans le rapport
- ✓ Langue : 4D lancé dans chaque langue cochée, sans déconnexion
- ✓ Test hors ligne : réseau coupé puis rétabli automatiquement
- ✓ Alerte « app téléchargée depuis Internet » validée automatiquement
- ✓ Rosetta testé sans intervention ; Command Line Tools facultatifs
- ✓ GitHub : jusqu'à 3 essais ; ⚠️ et non ❌ si GitHub est indisponible

Points d'attention restants

- Notarisation : il faut le certificat Developer ID de 4D (sinon install.sh)
- Accessibilité et Enregistrement de l'écran à réautoriser à chaque nouvelle version
- Pendant le test : session ouverte, écran non verrouillé

Pistes (à discuter en équipe)

- Signer et notariser l'app avec le certificat Developer ID de 4D
- Publier le code source dans un dépôt GitHub interne
- Un rapport unique Windows + macOS par release
- Valider les scénarios 4D Server et 4D Volume Desktop

6 · DÉMO ET QUESTIONS

Place à la démo

1. Ouvrir l’app : DMG détecté, version et build lus seuls
2. Prérequis au vert, cocher deux langues, lancer
3. Suivre le tableau : tout s’enchaîne sans intervention
4. Ouvrir le rapport HTML et les captures « À propos »
5. Montrer le ❌ du ReadMe (liens absents en DE)

À partager avec l’équipe

- **macOS :** TestInstaller4D.dmg
- **Windows :** TestInstaller4D.zip
- **Guide :** GUIDE_Test_Installer_4D.txt
- **Code source :** inclus, reconstructible

Merci ! Questions ?